

QUESTIONS 10 / 10 completed

Queue Using Array ▾

QUEUE USING ARRAYS

Write a C program to implement a Queue using Arrays with operations such as insertion (enqueue), deletion (dequeue), display, and peek. The program should simulate a ticket booking counter where customers arrive in sequence and

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int q[MAX], front = -1, rear = -1;
5
6 void enqueue(int x) {
7     if (rear == MAX - 1)
8         printf("Queue Overflow\n");
9     else {
10         if (front == -1) front = 0;
11         q[++rear] = x;
12     }
13 }
14
15 void dequeue() {
16     if (front == -1)
17         printf("Queue Underflow\n");
18     else {
19         printf("Deleted Element: %d\n", q[front]);
20         front++;
21         if (front > rear)
```

INPUT

```
1
101
1
102
1
103
3
2
3
5
```

QUESTIONS 10 / 10 completed

Queue Using Linke ▾

QUEUE USING LINKED LISTS

Write a C program to implement a Queue using Linked Lists for managing patients waiting in a hospital consultation room. The program should support enqueue, dequeue, display, and front operations without any fixed size limitation. Sample

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 struct Node *front = NULL, *rear = NULL;
10
11 void enqueue(int x) {
12     struct Node *newNode = malloc(sizeof(struct Node));
13     newNode->data = x;
14     newNode->next = NULL;
15
16     if (rear == NULL)
17         front = rear = newNode;
18     else {
19         rear->next = newNode;
20         rear = newNode;
21 }
```

INPUT

201
202
203
Display
Delete one patient

QUESTIONS 10 / 10 completed

Double Ended Que ▾

DOUBLE ENDED QUEUE

Write a C program to implement a Double Ended Queue (Deque) using Arrays where insertion and deletion can be performed from both front and rear ends. Consider a warehouse inventory management system where goods can be added or removed

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #define MAX 10
3
4 int dq[MAX], front = -1, rear = -1;
5
6 void insertFront(int x) {
7     if (front == 0)
8         printf("Overflow\n");
9     else {
10         if (front == -1)
11             front = rear = 0;
12         else
13             front--;
14         dq[front] = x;
15     }
16 }
17
18 void insertRear(int x) {
19     if (rear == MAX - 1)
20         printf("Overflow\n");
21     else {
```

INPUT

Insert Front 100
Insert Rear 200
Insert Front 50
Delete Rear

QUESTIONS 10 / 10 completed

Priority Queue Usir ▾

PRIORITY QUEUE USING ARRAYS

Write a C program to implement a Priority Queue using Arrays for processing emergency service requests. Requests with higher priority must be processed before lower-priority requests irrespective of arrival order.

Sample Input: Insert ▾

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #define MAX 10
3
4 int q[MAX], p[MAX], n = 0;
5
6 void insert(int x, int pr) {
7     q[n] = x;
8     p[n] = pr;
9     n++;
10 }
11
12 void delete() {
13     int i, pos = 0;
14
15     for (i = 1; i < n; i++)
16         if (p[i] < p[pos])
17             pos = i;
18
19     printf("Processed Request: %d\n", q[pos]);
20
21     for (i = pos; i < n - 1; i++) {
```

INPUT

101 3
102 1
103 2
Delete

QUESTIONS 10 / 10 completed

Queue Data Structu ▾

QUEUE DATA STRUCTURE

Write a C program to simulate a Railway Reservation Waiting List using Queue data structure. Passengers enter the waiting queue and are allocated seats as cancellations occur. The program should maintain passenger details and process

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #define MAX 10
3
4 int q[MAX], front = 0, rear = -1;
5
6 void add(int x) {
7     q[++rear] = x;
8 }
9
10 void display() {
11     int i;
12     printf("Passenger %d Confirmed.\n", q[front]);
13
14     if (front < rear) {
15         printf("Waiting List: ");
16         for (i = front + 1; i <= rear; i++)
17             printf("%d ", q[i]);
18     }
19 }
20
21 int main() {
```

INPUT

Add Passenger 501
Add Passenger 502
Cancellation Occurs
Display Queue

QUESTIONS 10 / 10 completed

Queue

QUEUE

Write a C program to implement a Queue for managing online food delivery orders where orders are received and processed in the order they arrive. The program should allow adding new orders, processing completed orders, and displaying

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2 #define MAX 10
3
4 int q[MAX], front = 0, rear = -1;
5
6 void add(int x) {
7     q[++rear] = x;
8 }
9
10 void process() {
11     printf("Processed Order: %d\n", q[front++]);
12 }
13
14 void display() {
15     int i;
16     printf("Pending Orders: ");
17     for (i = front; i <= rear; i++)
18         printf("%d ", q[i]);
19 }
20
21 int main() {
```

INPUT

Add Order 1001
Add Order 1002
Add Order 1003
Process Order

QUESTIONS 10 / 10 completed

Queue Using Linke ▾

QUEUE USING LINKED LISTS -2

Write a C program to implement a Queue using Linked Lists for managing network packets in a router. Packets should be transmitted in the order received, and the program must support dynamic packet insertion and removal. Sample Input: ▾

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 struct Node *front = NULL, *rear = NULL;
10
11 void insert(int x) {
12     struct Node *n = malloc(sizeof(struct Node));
13     n->data = x;
14     n->next = NULL;
15
16     if (rear == NULL)
17         front = rear = n;
18     else {
19         rear->next = n;
20         rear = n;
21     }
```

INPUT

11
12
13

QUESTIONS 10 / 10 completed

Circular Queue

CIRCULAR QUEUE

Write a C program to implement a Circular Queue for a traffic signal management system where vehicles are processed lane by lane in a cyclic manner. The system should continuously rotate through available lanes while maintaining queue

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int q[MAX], front = 0, rear = -1;
5
6 void enqueue(int x) {
7     rear = (rear + 1) % MAX;
8     q[rear] = x;
9 }
10
11 void dequeue() {
12     printf("Vehicle Passed: %d\n", q[front]);
13     front = (front + 1) % MAX;
14 }
15
16 void display() {
17     int i = front;
18     printf("Queue: ");
19     while (i != rear) {
20         printf("%d ", q[i]);
21         i = (i + 1) % MAX;
```

INPUT

21
22
23

QUESTIONS 10 / 10 completed

Queue-2

QUEUE-2

Write a C program to implement a Queue for managing examination answer sheet evaluation. Answer sheets submitted by students are evaluated in the same sequence in which they are received. Sample Input: Sheet IDs 401,

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #define MAX 10
3
4 int q[MAX], front = 0, rear = -1;
5
6 void enqueue(int x) {
7     q[++rear] = x;
8 }
9
10 void evaluate() {
11     printf("Evaluated Sheet: %d\n", q[front++]);
12 }
13
14 void display() {
15     int i;
16     printf("Pending Sheets: ");
17     for (i = front; i <= rear; i++)
18         printf("%d ", q[i]);
19 }
20
21 int main() {
```

INPUT

401
402
403